

# AMONG BOTS

[ILLUSTRATES.DEV](#)

[AMONG-BOTS-GITHUB](#)



Februar 9. 2026

**HFT STUTTGART**

[Schellingstraße 24,](#)  
[70174 Stuttgart](#)

Prepared By:

- **Akansha Darshi** – Logic Programming, Merging – GitHub
- **Hayder Abed** – ML Toxicity Model – GitHub
- **Maksym Pavlovskyi** – Docker Setup, Hosting – GitHub
- **Muhammed Natheer Abdullah** – AI Toxicity Model - GitHub
- **Roland Fazekas** – Logic Programming, AI Bots - GitHub
- **Ufuk Emre Sen** – UI Programming – GitHub

Datum: Februar 9. 2026

## Table of Contents

|                                    |    |
|------------------------------------|----|
| Summary.....                       | 3  |
| Project Objectives.....            | 3  |
| Scope of Services.....             | 3  |
| Stakeholder.....                   | 3  |
| Resources.....                     | 3  |
| Risk Management.....               | 3  |
| Communication.....                 | 4  |
| Quality Management.....            | 4  |
| Data Protection& Moderation.....   | 4  |
| Application Architecture.....      | 4  |
| Modules.....                       | 6  |
| Game Logic.....                    | 6  |
| Game State (Frontend).....         | 6  |
| Socket Events (Backend).....       | 7  |
| AI Player Logic.....               | 7  |
| Toxicity Filter.....               | 8  |
| Possible further developments..... | 10 |



## Summary

**Among Bots** is a multiplayer social deduction game in which real players compete in a group against LLM models as opponents. The goal is both entertainment and gathering insights and knowledge about AI deception in a controlled, playful environment.



## Project Objectives

- **Social Multiplayer Experience:** Development of a playable, web-based game specifically designed for groups of friends, as existing social dynamics enable the desired user experience.
- **AI:** Development of AI agents that mimic human behavior realistically enough to deceive human players within a closed lobby.
- **Orchestration:** Development of the application with a focus on easy deployment and scalability using Docker, accessible via common web browsers.

## Scope of Services

**In-Scope:** WEB: FrontEnd, BackEnd, AI integration, Containerized with Docker, Hosted on DigitalOcean(Droplet: VM), proxy with NGINX, DNS & Real-time Analytics with Cloudflare;

**Out-of-Scope:** Mobile apps (on IOS/Android), physical games.

## Stakeholder

Team Members, Users, Academic Supervisor

## Resources

**Team Roles:** Frontend, Backend, Basic DevOps and AI/ML.

**Tools:** TypeScript, React, [Node.js](#), [Express.js](#), Webpack(for UI build), [Socket.io](#), Docker, OpenAI API, GitHub.

## Risk Management

### Identified Risks:

- **API Costs:** Using external AI APIs (e.g., OpenAI) can incur significant costs, especially with high player or AI activity.

- **Toxic Content:** Player- or AI-generated content may contain toxic or inappropriate language.
- **AI Detection:** Human players can detect AI participants, which reduces game immersion.

#### **Mitigation strategies:**

- **Cost Control:** Use of filters and fallback logic to minimize unnecessary API calls
- **Content Moderation:** Integration of advanced toxicity filters and reinforcement learning from human feedback (RLHF) to reduce harmful outputs.
- **AI Camouflage:** Context-based behavior and output of AI agents to better mimic human players.



## **Communication**

Group chats und GitHub Pull requests

**Tools:** Slack, Discord, Zoom, GitHub, WhatsApp.

## **Quality Management**

Code quality via TypeScript.

Reviews via testing and controlled merges.

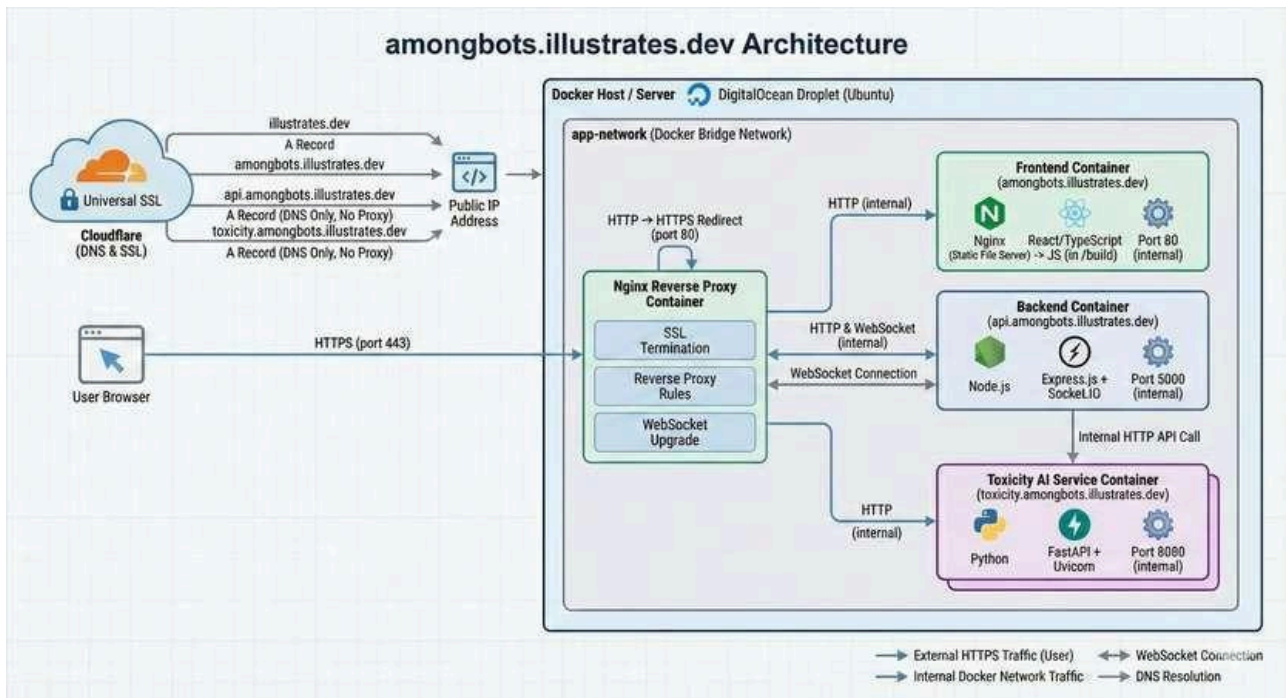
## **Datenschutz & Moderation**

**Data Protection:** Protection of user data through anonymity; no persistent user data is stored.

**Moderation:** Use of toxicity filters to detect and contain inappropriate content.

## **Application Architecture**

## Web:



## Frontend:

- Built with React and TypeScript.
- State management via Zustand.
- Communicates with the backend via Socket.IO for real-time updates.
- Stores minimal session data (Game code, Player ID) in localStorage for reconnection.

## Backend:

- Node.js Server with TypeScript.
- Real-time communication is handled by Socket.IO.
- Game state and Logic are managed in memory for each active game session.
- AI players are integrated as part of the game logic and can use external APIs (e.g., OpenAI) to generate responses, questions etc
- Toxicity filtering is performed using external machine learning models (Hugging Face Pipelines) that are accessible via an internal API.

## Date Flow:

- Clients senden Aktionen (join, submit, vote, etc.) an den Server.

- The Server validates, updates the game status, and sends changes to all clients in the game room.
- No persistent storage: All data is volatile and session-based.



## Modules

### Game Logik

- **Round Start:**
  - o startRoundForGame: Selects target player, creates prompt, sets participants.
- **Color Assignment:**
  - o When a player joins or is added, assignColor(game) assigns each player a unique color to avoid confusion.
- **Submission-Phase:**
  - o Timer for submissions, automatic placeholders for missing contributions.
- **Voting-Phase:**
  - o Timer for votes, AI votes are scheduled automatically.
- **Scoring:**
  - o Points for participation, bonus for quick submission, penalty for missing contributions.
  - o Players with 2+ missed submissions will be eliminated.
- **Elimination & Win-Conditions:**
  - o Players with the most votes are eliminated (except in the event of a tie).
  - o The game automatically detects when one side (AIs or humans) has won, e.g. when all humans have been eliminated or AIs are in the majority.
- **Fast-Track for AI-Votes:**
  - o Once all humans have voted, any remaining AI votes are triggered immediately (fastTrackAIVotesForRound).
- **Color Assignment**
  - o In the Backend (gameService.ts and colorPool.ts):

### Game State (Frontend)

#### State Storage with Zustand

In the Frontend (gameStore.ts) the current game status is managed with [Zustand](#).

- **Important Fields:**  
code, playerId, alias, colorId, isHost, gameState, roundNumber, lastGameSnapshot
- **Methods:**
  - o setFromCreateOrJoin(payload): Sets all fields after joining/creating a game.
  - o updateFromGame(game): Updates the status after a server update.
  - o reset(): Resets the store and removes local data.
- **Reconnect-Logic:**  
gameCode and playerId are stored in localStorage to enable reconnections after a connection failure or reload. The Backend Logic (game:reconnect in socket.ts) allows players to rejoin the game after a disconnect.



## Socket Events (Backend)

### Socket-Events and Game Flow

In the Backend (socket.ts), all game actions are handled via Socket.IO events:

- **game:create:** Creates a new game, sets up host, assigns color.
- **game:join:** Player joins, receives color and ID.
- **game:addAI:** Host can add AI players.
- **game:start:** Host starts the game, checks the minimum number of players, starts the first round.
- **round:submit:** Players submit entries.
- **round:vote:** Players vote.
- **game:reconnect:** Players can rejoin after disconnecting.
- **game:get:** Provides current game status.

Every change is sent to all clients in the room via emitGameUpdate.

- **Automatic AI Adjustment:**  
ensureAutoAIs feature dynamically adjusts the number of AI players to match the number of human players.

## AI Game Logic

### Integration and Behavior

In the Backend (aiPlayer.ts):

- **AI players are treated like human players**, but have additional fields (isAI, aiData).
- **Automated submission and voting:**

- o `scheduleAIForRound`: Schedules AI posts once all humans have submitted.
- o `scheduleAIVotesForRound`: Plans AI votes, with fallback mechanism.
- o **Toxicity-Filter**: AI only sees filtered content.
- **OpenAI-Integration:**
  - o AI can generate posts and make voting decisions using OpenAI models (e.g., GPT-4/5).
  - o Fallback: If no model is available, existing human contributions are copied.
- **Team Memory for AIs:**

AI players share notes and strategies via a shared team memory (`aiTeamMemory` in the game object).

This enables coordinated behavior and information sharing between AI agents.



## Toxicity Filter

### Classification within the overall project

In the Among Bots project, the AI Toxicity Model automatically analyzes chat messages from players and AI agents. The goal is to detect toxic behavior without mistakenly blocking statements typical of the game, such as suspicions or accusations.

- **Sanitizing & Caching:**

Toxicity ratings are cached to improve performance. The `sanitizeContentForAI` function replaces toxic content for AI players with placeholder text based on the model's rating.

### Research and model selection

The research was conducted on Hugging Face, a platform for pre-trained machine learning models.

Models used:

- `cardiffnlp/twitter-roberta-base-sentiment-latest`
- `MoritzLaurer/deberta-v3-large-zeroshot-v2.0`

### Model functions

- The sentiment model recognizes positive, neutral, and negative sentiments.
- The zero-shot model classifies texts using freely defined labels such as toxicity, accusation, or suspicion.

### Custom Labels

Separate labels have been defined for the game, which are specifically tailored to social deduction gameplay:

```
CUSTOM_LABELS = [
```

```
    "toxicity",
```

```
    "insult",
```

```
    "harassment",
```

```
    "threat",
```

```
    "hate_speech",
```

```
    "non_toxic",
```

```
    "accusation",
```

```
    "suspicion",
```

```
]
```



## Technical implementation

Both models are loaded using the Hugging Face pipeline API. This eliminates the need for manual tokenization or model logic.

The central function `classify_toxicity` combines both models: `def`

```
classify_toxicity(text):
```

```
    sentiment = sentiment_pipe(text)[0]
```

```
    zs = zs_pipeline(text, custom_labels, multi_label=True)
```

```
    return {
```

```
        "input": text,
```

```
        "sentiment_label": sentiment["label"],
```

```
        "sentiment_score": float(sentiment["score"]),
```

```
        "toxicity_scores": {
```

```
            label: float(score)
```

```
    for label, score in zip(zs["labels"], zs["scores"])\n\n    }\n\n}
```

## Possible further developments



**Expansion of game modes and rounds:** Introduction of additional discussion phases before or after voting, in which each player can write their own contribution. Supplementary rounds in which players give each other topics or writing instructions, thereby prompting both other players and the AI in a targeted manner. Other round formats could include drawing tasks, image descriptions, or other non-textual interactions.

**Finalization of the Website:** Further development into a more visually appealing website with a landing page and improved protection mechanisms against unstable or poor network connections.